

Power of a Number — DAA

This implementation calculates a^n using divide and conquer (exponentiation by squaring). Dividing the exponent by 2 at every step makes it much faster than multiplying the base n times.

Algorithm

1. If exponent is 0, return 1.
2. Divide the exponent by 2.
3. Find the power of the smaller exponent recursively.
4. If exponent is even, square the result.
5. If exponent is odd, multiply the result by the base as well.
6. Return the final result.

C Code

```
#include <stdio.h>

long long power(int base, int exponent)
{
    long long half;

    if (exponent == 0)
        return 1;

    half = power(base, exponent / 2);

    if (exponent % 2 == 0)
        return half * half;
    else
        return base * half * half;
}

int main()
{
    int base, exponent;

    printf("Enter base: ");
    scanf("%d", &base);

    printf("Enter exponent: ");
    scanf("%d", &exponent);

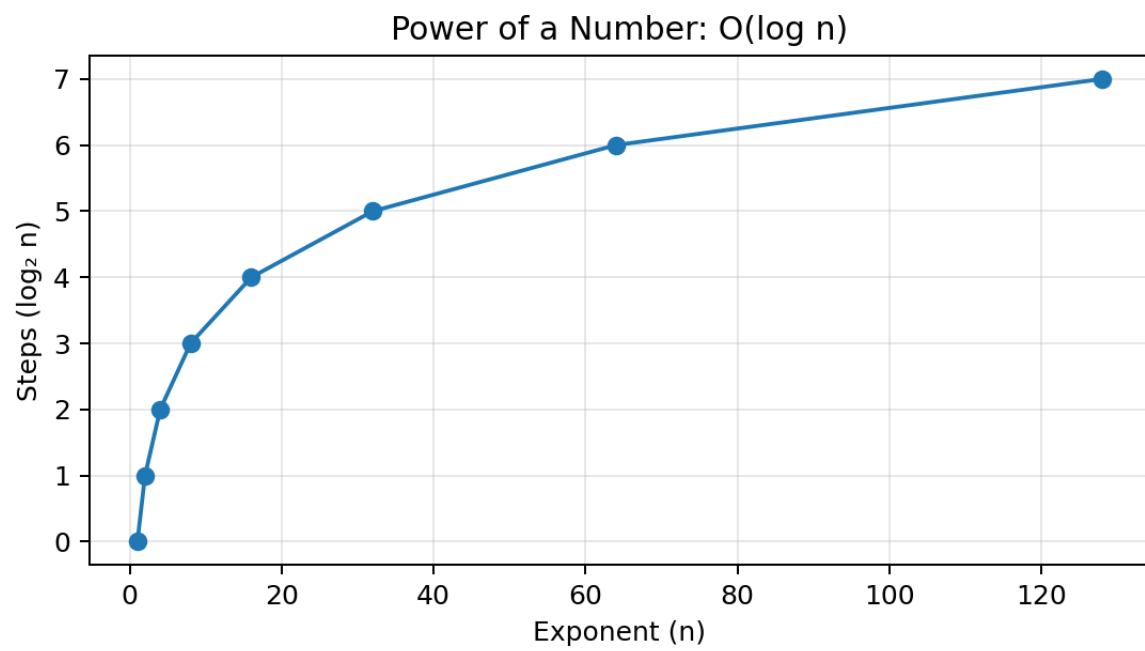
    printf("%d raised to the power %d = %lld",
           base, exponent, power(base, exponent));

    return 0;
}
```

Time Complexity

Time Complexity: $O(\log n)$

The exponent is divided by 2 at every recursive call, so the number of steps grows logarithmically.



Example Output

```
Enter base: 2
Enter exponent: 5
2 raised to the power 5 = 32
```

Viva / Exam Point

This is a **divide-and-conquer** approach. The key idea is to repeatedly reduce the exponent by half.